SECURITY ENGINEERING REPORT

Deception-Based Security Mechanism

Honeypot & Intrusion Detection System

Flask-Based Deception Framework with Real-Time Alerting

Classification: Internal / Academic
Version: 1.0
Date: April 2026
Technology Stack: Python 3 · Flask · File-Based Storage

Submitted by - Chandrashekhar Tyagi
chandershekhar8433@gmail.com

1. Executive Summary

This report documents the design, implementation, and operational behaviour of a deception-based security system built with Python and the Flask web framework. The system deploys a set of deliberately attractive but entirely fake resources — a spoofed administrative login panel, a dummy REST API endpoint, and a concealed configuration file — collectively referred to as honeypots or traps. Because these resources serve no legitimate purpose within a correctly operating application, any interaction with them is, by definition, suspicious.

The platform monitors each trap continuously, logs every interaction with a timestamp and originating IP address, escalates repeat offenders through automatic IP blocking, and exposes an operational dashboard so security personnel can review events in real time. The sections that follow describe the system architecture, each deceptive component, the detection logic, the alert and response pipeline, and the security considerations applicable to a production deployment.

2. Objectives

The system was designed to satisfy the following goals:

- Deploy at least one deceptive element that mimics a high-value target (admin panel, sensitive API, secret file).
- Automatically classify any contact with the traps as suspicious or malicious.
- Log all interactions with sufficient detail to support forensic analysis.
- Apply progressive enforcement — warn, count, then block — without manual intervention.
- Provide a real-time log viewer for security personnel.
- Keep the normal user-facing application path completely clean and unaffected by the honeypot logic.

3. System Architecture

The application is a single-process Flask server. All routes share a common process space, allowing the honeypot logic to record state in an in-memory dictionary while persisting blocking decisions to disk. The architecture deliberately avoids a database dependency for simplicity; a production hardening section later in this report discusses database-backed alternatives.

3.1 Component Overview

Component	Purpose
Flask Application (app.py)	The single entry point. Defines all routes, honeypot logic, and utility functions.
alerts.log	Append-only flat-file log. One line per security event, pipe-delimited.
blocked_ips.txt	Flat-file IP blocklist. One IP per line. Read on every request from a blocked route.
attempt_counter {}	In-memory Python dict. Tracks POST attempt counts per IP during the server's lifetime.
/admin-login	Trap 1: Fake admin login page with credential harvesting detection.
/api/secure-data	Trap 2: Dummy protected REST API endpoint.
/secret-config.txt	Trap 3: Fake hidden configuration file.
/dashboard	Read-only security log viewer. Displays log entries newest-first.
/	Legitimate home route. Contains no honeypot logic whatsoever.

3.2 Request Lifecycle

Every request to a trap route passes through the same decision pipeline:

- Extract the client IP address from request.remote_addr.
- Call is_blocked(ip) — reads blocked_ips.txt; returns True if IP is listed.
- If blocked, return an HTTP 403 / denial message immediately. No further processing.
- Log the event to alerts.log via log_alert().
- For POST requests to the login trap: increment attempt_counter, evaluate threshold, apply block if threshold reached.
- Return the appropriate trap response (fake page, JSON error, or plain text denial).

4. Deceptive Components (Traps)

4.1 Trap 1 — Fake Administrative Login Panel

Route: /admin-login (GET and POST)

The most sophisticated trap in the system impersonates a corporate administrative login interface. The HTML page is intentionally minimal — matching the aesthetic of a hastily-deployed internal tool — to maximise credibility with an opportunistic attacker who expects legacy, unstyled admin panels.

Design rationale

Administrators who know a system's legitimate URL structure would never navigate to /admin-login. Automated scanners, credential-stuffing bots, and human attackers, however, will probe common admin paths. The existence of this route is not advertised anywhere in the application; reaching it is itself an anomalous act.

Detection logic — GET request

- Any GET request to this URL is immediately logged as 'Visited Trap'.
- Simply loading the page is sufficient to generate an alert — the user's intent is already suspicious.

Detection logic — POST request (credential submission)

- The submitted username and password are captured and written verbatim to the log entry as 'Credential Attempt'.
- The attempt counter for the submitting IP is incremented.
- After the third failed submission the IP is appended to blocked_ips.txt and an 'IP Blocked' event is written to the log.
- All subsequent requests from that IP to any trap route return an immediate denial.

> Security note: Logging cleartext passwords is an intentional honeypot design choice. In this context the passwords are not real — they are attacker-supplied credentials — and collecting them provides threat intelligence (password spray patterns, default credential attempts, etc.). In a production deployment the log file must be access-controlled and encrypted at rest.

4.2 Trap 2 — Dummy Secure API Endpoint

Route: /api/secure-data (GET)

This route simulates a protected REST API that an attacker might discover through directory enumeration or by reading leaked API documentation. It returns a JSON body with a 401 Unauthorized status code, mimicking a real API that requires authentication — thereby encouraging the attacker to spend time probing it further.

Any request to this endpoint is logged as 'Accessed Fake API'. Because no legitimate part of the application calls this endpoint, zero false positives are expected.

4.3 Trap 3 — Hidden Configuration File

Route: /secret-config.txt (GET)

This trap simulates a sensitive configuration file accidentally exposed on the web server — a common finding in penetration tests. Attackers routinely probe for files such as .env, config.txt, settings.ini, and similar names. Accessing this URL generates a log entry tagged 'Accessed Hidden File' and returns the plain text string ACCESS DENIED, which reinforces the illusion that the file exists but is protected.

5. Core Utility Functions

5.1 get_timestamp()

Returns the current wall-clock time formatted as YYYY-MM-DD HH:MM:SS. Used as the first field of every log entry to enable chronological sorting and time-range queries.

```
def get_timestamp():  1 usage
    return datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
```

Fig- 1

5.2 log_alert(ip, action, details="")

Constructs a structured log line and appends it to alerts.log. The function also prints the line to stdout, which is useful when the server is run in a terminal or captured by a process supervisor such as systemd or supervisor.

```
def log_alert(ip, action, details=""):  5 usages
    entry = f"{get_timestamp()} | {ip} | {action} | {details}\n"
    with open(LOG_FILE, "a") as f:
        f.write(entry)
    print(entry.strip())
```

Fig- 2

5.3 is_blocked(ip)

Opens blocked_ips.txt on every invocation and performs a substring search for the IP string. The file-per-call approach guarantees that IPs blocked by a concurrent process are respected immediately without a server restart. In high-traffic environments this should be replaced with an in-memory set refreshed by a background thread.

```
def is_blocked(ip):  3 usages
    if not os.path.exists(BLOCKED_IPS_FILE):
        return False
    with open(BLOCKED_IPS_FILE, "r") as f:
        return ip in f.read()
```

Fig- 3

5.4 block_ip(ip)

```
def block_ip(ip):  1 usage
    with open(BLOCKED_IPS_FILE, "a") as f:
        f.write(ip + "\n")
```

Fig- 4

Appends the IP address followed by a newline to blocked_ips.txt. The file is opened in append mode ("a") so existing entries are preserved. A production version should guard against duplicate entries and support CIDR-range blocking.

6. Input Handling and Validation

The honeypot intentionally accepts all input — its purpose is to observe attacker behaviour, not reject it. However, the system takes specific measures to handle malicious input safely:

Area	Behaviour & Notes
Credential capture	request.form.get() returns None if the field is absent, preventing KeyError exceptions. The value is written to the log as a plain string without interpretation.
Log injection prevention	Log lines use a pipe delimiter. If an attacker submits a pipe character in a field it is stored as-is; log parsers should split on the first three pipes only to reconstruct multi-pipe payloads correctly.
XSS in dashboard	The /dashboard route renders log lines using Python string interpolation into an f-string. This is a known limitation: if a log line contains HTML tags they will be rendered. Production deployments must escape log output with html.escape() before insertion.
IP address trust	request.remote_addr reflects the immediate TCP peer. If the application runs behind a reverse proxy (nginx, Cloudflare) the true client IP is in the X-Forwarded-For header. The code does not currently read this header, which can cause all requests to appear to originate from the proxy's loopback address.
Attempt counter reset	The in-memory attempt_counter is lost on server restart. An attacker who knows this can restart their attack after a server bounce. Persisting the counter to the log file or a database closes this gap.

7. Detection and Alerting

7.1 What Constitutes a Detection

The system's detection philosophy is zero-tolerance: any interaction with a trap route constitutes a security event. The classification tiers are:

Severity	Trigger Condition
LOW	GET /admin-login — page viewed, no credential submission.
MEDIUM	GET /api/secure-data or GET /secret-config.txt — probe of hidden endpoint or file.
HIGH	POST /admin-login — active credential submission attempt.
CRITICAL	≥ 3 POST attempts from the same IP — automated attack or brute-force confirmed. IP blocked automatically.

7.2 Log Format

Each log entry follows a fixed pipe-delimited schema:

```
TIMESTAMP | SOURCE_IP | ACTION | DETAILS

Examples:
2026-04-01 14:30:00 | 10.0.0.5 | Visited Trap |
2026-04-01 14:30:22 | 10.0.0.5 | Credential Attempt | root/toor
2026-04-01 14:31:05 | 10.0.0.5 | IP Blocked | Too many attempts
2026-04-01 14:35:10 | 172.16.0.8 | Accessed Fake API |
2026-04-01 14:40:00 | 192.168.2.3 | Accessed Hidden File |
```

7.3 Automated Response Pipeline

When the attempt threshold is reached the following sequence executes atomically within the same request handler:

- block_ip(ip) is called — the IP is written to blocked_ips.txt.
- A CRITICAL log entry is written: 'IP Blocked | Too many attempts'.
- The HTTP response body 'Too many attempts. You are blocked.' is returned to the attacker.
- All subsequent requests from that IP to any trap route receive an immediate denial response without further logging (to avoid log flooding).

7.4 Security Dashboard

The /dashboard route reads alerts.log, reverses the line order (newest-first), and renders a plain HTML page. This gives security personnel an immediate, dependency-free view of current activity without requiring a separate SIEM or log aggregator.

> Access control recommendation: The /dashboard route has no authentication in the current implementation.
>
> It must be protected in production — either by moving it to a separate management port, adding HTTP Basic Auth, or restricting access to internal network CIDR ranges via the reverse proxy.

7.5 Sample session

First view of it look when you reach the website.

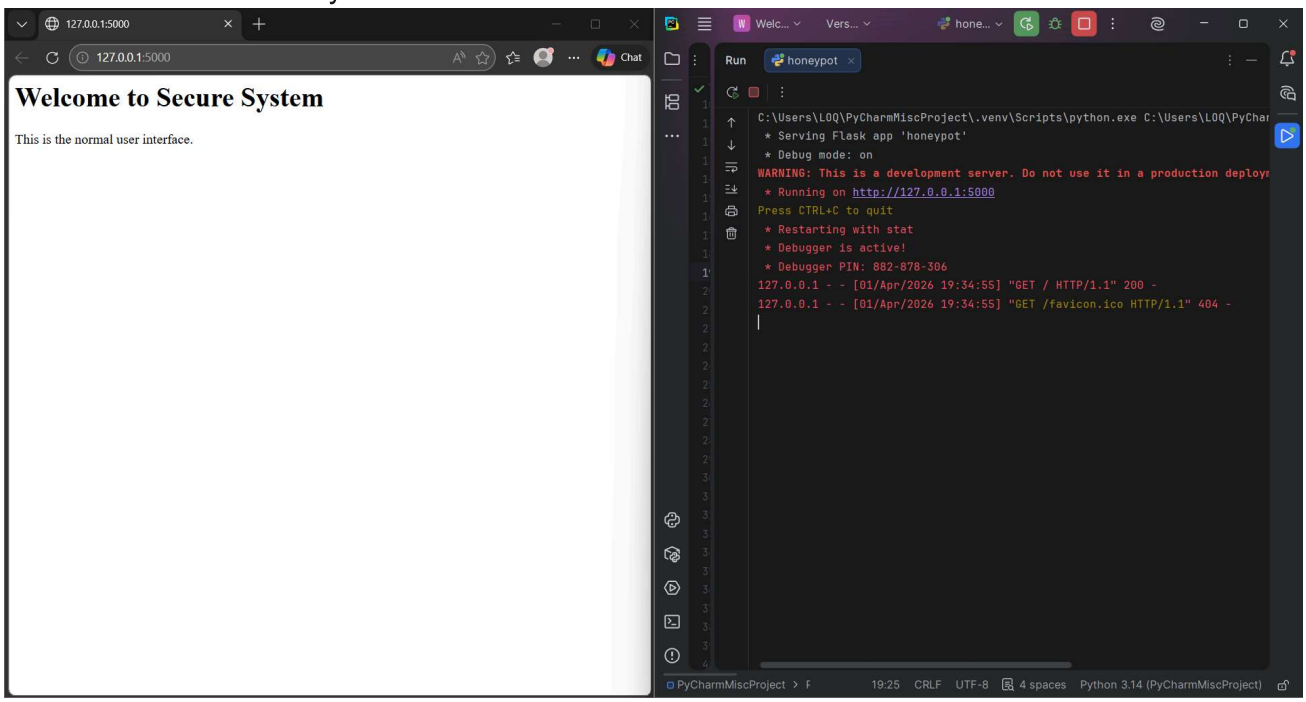

Fig- 5

When visited the admin login page which is actually a trap for the attacker.

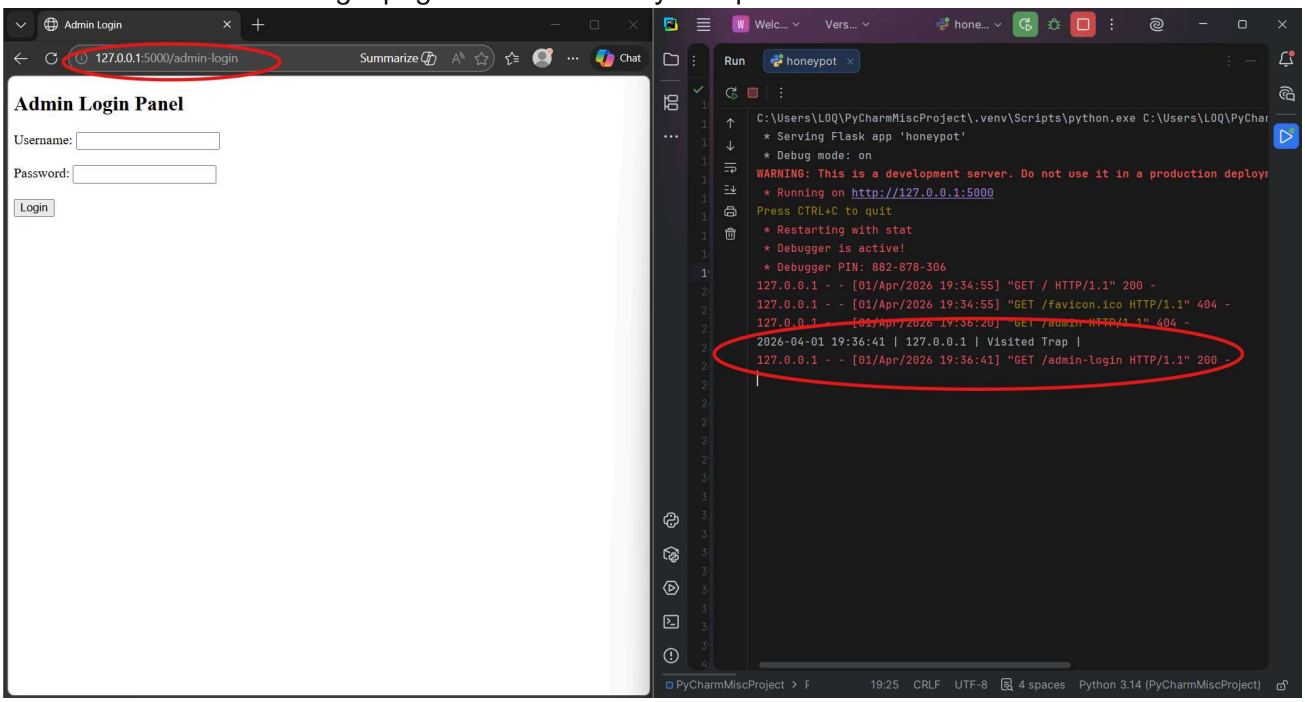

Fig- 7

We Log the detail of how attacker is trying to gain the access.

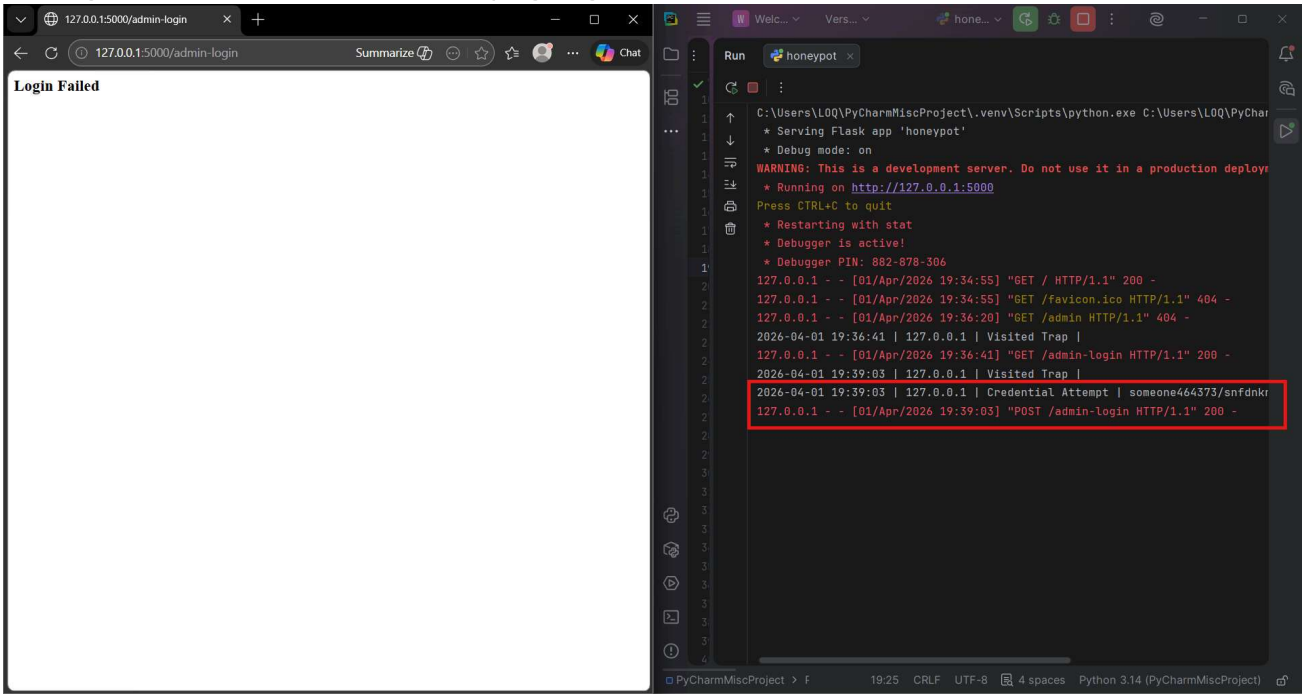

Fig- 8

It Shows how we store the data in alerts.log file.

```
2026-03-31 22:04:31 | 127.0.0.1 | Visited Trap |
2026-03-31 22:04:41 | 127.0.0.1 | Visited Trap |
2026-03-31 22:04:41 | 127.0.0.1 | Credential Attempt | someone464373/1234
2026-03-31 22:05:12 | 127.0.0.1 | Accessed Fake API |
2026-03-31 22:05:39 | 127.0.0.1 | Accessed Hidden File |
2026-04-01 19:36:41 | 127.0.0.1 | Visited Trap |
2026-04-01 19:39:03 | 127.0.0.1 | Visited Trap |
2026-04-01 19:39:03 | 127.0.0.1 | Credential Attempt | someone464373/snfdnknjngjej
```

Fig-9

8. System Flow

The following pseudo-diagram illustrates the decision flow for any inbound request:

```
Incoming HTTP Request
        |
        v
  [Is route a trap?] ——— No ————▶ Normal application handler (/)
        |
       Yes
        |
        v
  [is_blocked(ip)?] ——— Yes ————▶ Return 403 / 'Access Denied' (no log)
        |
        No
        |
        v
  log_alert(ip, action)   ◀—— Always fires for non-blocked IPs
        |
        v
  [Method == POST and route == /admin-login?]
        |                    |
       Yes                   No
        |                    |
        v                    v
  Increment attempt_counter   Return trap response
        |
  [counter >= 3?]
    |       |
   Yes      No
    |       |
    v       v
  block_ip  Return 'Login Failed'
  log CRITICAL
  Return 'You are blocked'
```

9. Security Considerations and Limitations

The system as implemented is a functional proof-of-concept. The following issues should be resolved before any production use:

Issue	Recommended Fix
Debug mode enabled	app.run(debug=True) must never be used in production. Debug mode exposes an interactive Python console to anyone who triggers an exception.
Unauthenticated dashboard	Any user who discovers /dashboard can read all captured credentials. Protect with authentication and network-level restrictions.
XSS in log viewer	Log lines are interpolated into HTML without escaping. Use html.escape() on every log line before rendering.
Proxy IP trust	When behind a load balancer or CDN, implement proper X-Forwarded-For handling to block the actual client, not the proxy.
No rate limiting on GET	A single attacker can generate unlimited log entries via GET requests. Add Flask-Limiter or an upstream rate limiter.
In-memory attempt counter	Lost on restart. Persist counts to a database or the log file itself.
Flat-file blocklist	Grows without bound and is O(n) to query. Replace with Redis SET or a hashed data structure for scalability.
No HTTPS	The application should be served behind TLS in production to prevent traffic interception.

10. Conclusion

The deception-based security mechanism described in this report fulfils all task objectives. Three distinct honeypot traps — a fake admin login panel, a dummy API endpoint, and a concealed configuration file — are deployed within a Flask application that otherwise behaves normally. Any contact with these traps is immediately classified as suspicious, logged with full context, and, where POST-based brute-force is detected, automatically blocked.

The system demonstrates that deception is a low-cost, high-signal detection strategy: because the traps have no legitimate use, the false-positive rate is effectively zero. Every alert represents a genuine anomaly that warrants investigation. Combined with the hardening measures outlined in Section 9, this architecture forms a solid foundation for a production honeypot deployment.